

MANUAL DEL INTEGRANTE 4 — FRONTEND (JAVA FX)

Proyecto

Sistema de diseño y optimización de redes de fibra óptica utilizando algoritmos voraces, JavaFX y PostGIS.

1. OBJETIVO DEL MÓDULO

Este módulo será responsable de:

- Mostrar mapa.
 - Mostrar calles.
 - Mostrar nodos.
 - Mostrar conexiones.
 - Mostrar resultados de algoritmos.
 - Permitir interacción del usuario.
 - Mostrar leyenda visual.
 - Convertir eventos del usuario en llamadas al Backend.
-

NO ES RESPONSABILIDAD DEL FRONTEND

No deberá implementar:

- SQL
- PostgreSQL
- PostGIS
- Prim
- Kruskal
- Dijkstra
- Cálculo de costos
- Validaciones espaciales
- Saturación de postes

Todo eso pertenece a Backend.

2. ESTRUCTURA DEL PROYECTO

```
frontend/
├── controllers/
│   ├── MainController.java
│   ├── MapController.java
│   ├── ToolbarController.java
│   └── LegendController.java
├── views/
│   ├── MainView.fxml
│   ├── MapView.fxml
│   └── ToolbarView.fxml
├── map/
│   ├── MapRenderer.java
│   ├── NodoVisual.java
│   ├── ConexionVisual.java
│   └── LegendPanel.java
└── styles/
    └── application.css
```

3. FLUJO GENERAL

```
Usuario
↓
Frontend
↓
Backend 2
↓
Backend 1
↓
Resultado
↓
Frontend
↓
Mapa
```

4. CLASE MainController

Ubicación:

```
controllers/MainController.java
```

Responsabilidad:

Control principal de la aplicación.

MÉTODO

initialize()

```
public void initialize()
```

Retorna:

```
void
```

FUNCIÓN

Inicializar:

- mapa
 - herramientas
 - leyenda
-

MÉTODO

cargarProyecto()

```
public void cargarProyecto()
```

Retorna:

```
void
```

FUNCIÓN

Solicitar al Backend:

```
calleService.obtenerCalles();  
nodoService.listarNodos();  
conexionService.listarConexiones();
```

5. CLASE ToolbarController

Ubicación:

```
controllers/ToolbarController.java
```

RESPONSABILIDAD

Gestionar botones.

BOTONES

```
Central  
Poste  
Cliente  
Generar Red  
Comparar Algoritmos  
Eliminar Nodo
```

ATRIBUTO

```
private TipoNodo modoActual;
```

MÉTODO

seleccionarCentral()

```
public void seleccionarCentral()
```

Retorna:

```
void
```

Implementación:

```
modoActual = TipoNodo.CENTRAL;
```

MÉTODO

seleccionarPoste()

```
public void seleccionarPoste()
```

Retorna:

```
void
```

Implementación:

```
modoActual = TipoNodo.POSTE_SECUNDARIO;
```

MÉTODO

seleccionarCliente()

```
public void seleccionarCliente()
```

Retorna:

```
void
```

Implementación:

```
modoActual = TipoNodo.CLIENTE;
```

MÉTODO

generarRed()

```
public void generarRed()
```

Retorna:

```
void
```

FUNCIÓN

Mostrar diálogo:

Seleccione algoritmo:

- () Prim
- () Kruskal

Según selección:

```
networkService.generarRed(  
    grafo,  
    central,  
    algoritmo  
);
```

MÉTODO

compararAlgoritmos()

```
public void compararAlgoritmos()
```

Retorna:

```
void
```

FUNCIÓN

Llamar:

```
networkService.compararAlgoritmos(  
    grafo,  
    central  
);
```

Mostrar:

```
Costo total  
Distancia total  
Cantidad de postes
```

para ambos algoritmos.

6. CLASE MapController

Ubicación:

```
controllers/MapController.java
```

RESPONSABILIDAD

Gestionar clics sobre el mapa.

ATRIBUTO

```
private TipoNodo modoActual;
```

MÉTODO

onMapClick()

```
public void onMapClick(  
    double latitud,  
    double longitud  
)
```

Retorna:

```
void
```

FUNCIÓN

Crear:

```
CENTRAL  
POSTE  
CLIENTE
```

según modo seleccionado.

IMPORTANTE

El Frontend NO valida.

Simplemente llama:

```
nodoService.crearNodo(dto);
```

Si Backend responde error:

```
SpatialException
```

mostrar alerta.

ESCENARIO ESPECIAL

Conversión de Poste Sugerido

Si el usuario:

```
Seleccionar Poste  
↓  
Click sobre nodo sugerido
```

Entonces:

```
nodoService.crearNodo(dto);
```

utilizando las coordenadas del nodo sugerido.

Resultado:

```
SUGERIDO  
↓  
POSTE_SECUNDARIO
```

7. CLASE MapRenderer

Ubicación:

```
map/MapRenderer.java
```

RESPONSABILIDAD

Dibujar elementos visuales.

MÉTODO

dibujarCalles()

```
public void dibujarCalles(  
    List<CalleDTO> calles  
)
```

Retorna:

```
void
```

MÉTODO

dibujarNodos()

```
public void dibujarNodos(  
    List<NodoDTO> nodos  
)
```

Retorna:

```
void
```

MÉTODO

dibujarConexiones()

```
public void dibujarConexiones(  
    List<ConexionDTO> conexiones  
)
```

Retorna:

```
void
```

8. CLASE NodoVisual

Ubicación:

```
map/NodoVisual.java
```

RESPONSABILIDAD

Representar un nodo gráficamente.

ATRIBUTOS

```
private NodoDTO nodo;
```

MÉTODO

actualizarColor()

```
public void actualizarColor()
```

Retorna:

```
void
```

IMPORTANTE

NO calcular saturación.

Solo interpretar:

```
nodo.getEstado()
```

COLORES POR TIPO

Tipo	Color
CENTRAL	Azul
POSTE	Verde
CLIENTE	Rojo
SUGERIDO	Amarillo

COLORES POR ESTADO

Estado	Color
DISPONIBLE	Normal
SATURADO	Naranja
INACTIVO	Gris

EJEMPLO

Si Backend envía:

```
estado = SATURADO
```

Mostrar:

Naranja

9. CLASE ConexionVisual

Ubicación:

map/ConexionVisual.java

RESPONSABILIDAD

Representar conexiones.

ATRIBUTOS

```
private ConexionDTO conexion;
```

MÉTODO

dibujar()

```
public void dibujar()
```

Retorna:

```
void
```

10. CLASE LegendPanel

Ubicación:

map/LegendPanel.java

RESPONSABILIDAD

Mostrar significado de símbolos.

POSICIÓN

Esquina inferior derecha.

CONTENIDO

-  Central
-  Poste
-  Cliente
-  Poste sugerido
-  Poste saturado
-  Poste inactivo

IMPORTANTE

La leyenda debe permanecer visible todo el tiempo.

11. PANEL DE INFORMACIÓN

Al hacer clic sobre un nodo mostrar:

Nombre
Tipo
Estado
Clientes actuales
Capacidad máxima
Capacidad disponible

DATOS

Proviene de:

NodoDT0

NO CALCULAR

No realizar:

capacidadMax - clientesActuales

Utilizar:

capacidadDisponible

si Backend lo envía.

12. ALERTAS

Mostrar alertas para:

Poste fuera de calle

No existe central

No existen clientes

No se pudo generar la red

Error de conexión

13. COMUNICACIÓN CON BACKEND

CREAR NODO

```
nodoService.crearNodo(dto);
```

LISTAR NODOS

```
nodoService.listarNodos();
```

LISTAR CALLES

```
calleService.obtenerCalles();
```

LISTAR CONEXIONES

```
conexionService.listarConexiones();
```

GENERAR RED

```
networkService.generarRed(  
    grafo,  
    central,  
    algoritmo  
);
```

COMPARAR ALGORITMOS

```
networkService.compararAlgoritmos(  
    grafo,  
    central  
);
```

14. GITHUB

Rama asignada:

frontend

REGLAS

- No modificar Backend 1.
- No modificar Backend 2.
- No modificar Base de Datos.
- No implementar algoritmos.
- No implementar SQL.
- Mantener interfaz limpia.
- Documentar componentes visuales.

15. OBJETIVO FINAL

Al finalizar este módulo deberá ser posible:

- Visualizar calles.
- Crear centrales.
- Crear postes.
- Crear clientes.
- Visualizar conexiones.
- Generar redes.
- Comparar Prim y Kruskal.
- Mostrar postes sugeridos.
- Convertir sugeridos en postes reales.
- Mostrar estados mediante colores.
- Mostrar leyenda permanente.
- Mostrar información detallada de nodos.
- Integrarse completamente con Backend y Base de Datos.